AMERICAN UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF ENGINEERING AND COMPUTER SCIENCE



STEELCUT OPTIMIZER SYSTEM

Prepared By

Ahmad Abou Ilawi

Lama Al Sayed

Shaza Hamzeh

Hakima Bazzal

Submitted to

Dr. Hasnaa Hussein Sleiman

Zahle, Lebanon

Spring 2026

Abstract

This project presents the design and implementation of a C++ console-based optimization system for steel plate cutting operations. The Steel Cut Optimizer System was developed to address the common industrial challenge of material waste during the cutting of steel plates into required pieces. The system minimizes material waste by systematically evaluating all available stock plates and selecting the most efficient plate for each cutting task.

The program supports piece rotation, enabling the optimizer to test both normal and rotated orientations when fitting required pieces onto available plates. It integrates kerf-aware formulas to account for blade thickness during fit calculations, ensuring accurate and realistic results. Comprehensive validation checks are embedded throughout the input process to prevent erroneous data from affecting the optimization logic.

The system generates instant financial quotations by computing the total cost based on the number of plates required and user-defined pricing parameters. The efficiency percentage and waste area are also calculated and presented in a clearly formatted output. By automating these computations, the Steel Cut system reduces manual calculation errors, improves productivity, and provides a structured and reliable workflow for industrial cutting operations. The solution was implemented using ISO C++ with static arrays and structured control flow, compiled using the GCC/MinGW build chain.

© American University of Science & Technology - 2022

Table of Contents

I. Introduction 1
II. Objectives 1
III. Background 2
IV. Design and Procedure 4
A. System Design 4
B. Step by Step Procedure 4
1. Step 1: Input Collection and Validation 5
2. Step 2: Optimization Loop 6
3. Step 3: Output and Invoice Generation 7
C. Cost Description 8
V. Problems Faced and Solutions 9
VI. Results and Conclusions 10
A. Results 10
B. Conclusions 10

List of Figures

Figure 1. Program input phase: welcome banner, kerf and plate data entry
Figure 2. System architecture overview and input/output flow diagram
Figure 3. Program execution flowchart
Figure 4. Optimization results console output
Figure 5. Input validation: negative kerf error and recovery
Figure 6. Financial cost and invoice module output

List of Tables

Table 1. Cost description

I. Introduction

Steel plate cutting is a fundamental industrial process used in manufacturing, construction, and fabrication industries. When cutting steel plates into required pieces, inefficient material usage and excessive waste are recurring problems that directly impact operational costs. Manual approaches to plate selection and waste calculation are often time-consuming, error-prone, and fail to account for variables such as blade kerf thickness, piece rotation flexibility, and stock inventory limits.

The Steel Cut Optimizer System was developed as a software solution to address these challenges. This project presents a C++ console-based application that automates the process of plate selection, waste minimization, efficiency calculation, and financial quotation generation. The system accepts the blade kerf value, available plate dimensions and quantities, required piece dimensions, and cost parameters as inputs, then systematically evaluates all available plate options to recommend the most efficient choice. By integrating kerf-aware formulas, rotation support, stock sufficiency enforcement, and a complete invoicing module, the Steel Cut system provides a comprehensive and accurate tool for industrial cutting operations [1].

II. Objectives

The primary objective of this project is to design and implement a reliable C++ console application that optimizes the selection of steel plates for cutting operations. The system aims to achieve the following specific goals:

1. Minimize material waste by selecting the plate with the lowest waste area among all feasible options.
2. Support piece rotation to improve fitting flexibility and plate utilization efficiency.

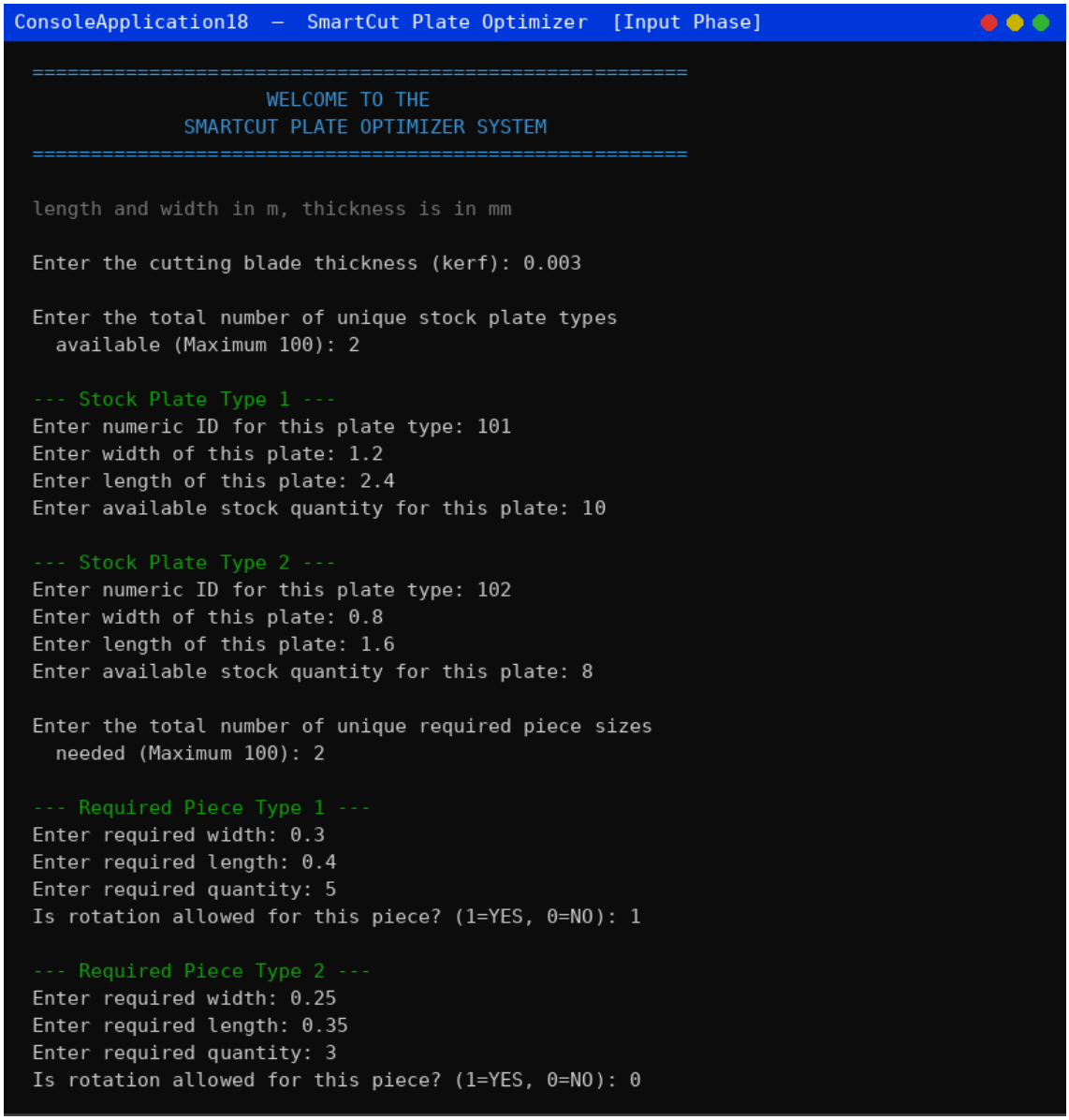
3. Integrate kerf-aware fit formulas to account for blade thickness and produce accurate cutting calculations.
4. Enforce stock sufficiency to ensure the recommended plate has adequate available quantity.
5. Generate an automated financial invoice based on user-defined material and labor pricing parameters.
6. Provide a clear, structured, and user-friendly console interface for industrial operators.

III. Background

Cutting stock problems are a well-studied class of optimization problems in operations research and combinatorial mathematics. The general problem involves cutting raw material—in this case, rectangular steel plates—into a set of required smaller pieces while minimizing waste. This class of problems has wide industrial applications in steel fabrication, glass cutting, textile manufacturing, and paper production [2].

Traditional industrial approaches rely on manual calculations or basic spreadsheet-based tools to determine plate usage and estimate waste. These methods are limited in their ability to consider multiple variables simultaneously, such as blade kerf thickness, piece orientation, stock availability, and cost optimization. Software-based solutions, particularly those implemented in compiled languages such as C++, offer significant advantages in terms of speed, accuracy, and scalability [2].

Figure 1. Program input phase: welcome banner, kerf and plate data entry.



The kerf value refers to the width of material removed by the cutting blade during each cut. When fitting multiple pieces onto a plate, the kerf must be factored into the calculations for each internal cut to avoid underestimation of space requirements. Piece rotation, where a piece's length and width are swapped, can significantly improve plate utilization in cases where the rotated orientation allows more pieces to fit on a given plate [2].

IV. Design and Procedure

The Steel cut Optimizer System was designed using a structured procedural approach in ISO C++. The program is organized into distinct logical modules covering input collection, validation, optimization, and output generation [3].

A. System Design

The system is structured around a main processing loop that accepts user inputs, validates them, and evaluates each available plate against a set of defined criteria. The inputs include blade kerf thickness, stock plate details (ID, width, height, quantity, price per plate), required piece dimensions (width, height, quantity), rotation flags, and cost parameters (labor cost per plate and profit margin percentage).

For each plate in the stock array, the system computes the maximum number of pieces that fit using kerf-adjusted formulas for both normal and rotated orientations. The plate that satisfies the quantity requirement with minimum waste is selected as the optimal choice. If multiple plates yield equal waste, the one with the higher efficiency percentage is preferred. The overall system architecture and input/output flow are illustrated in Figure 2 [4].

Figure 2. System architecture overview and input/output flow diagram.

The key formula for computing the number of pieces fitting along one plate dimension, while accounting for kerf, is: count = floor((P + K) / (D + K)), where P is the plate dimension, D is the piece dimension, and K is the kerf thickness. This formula correctly accounts for the fact that the last piece in a row does not require a trailing kerf cut [4].

B. Step by Step Procedure

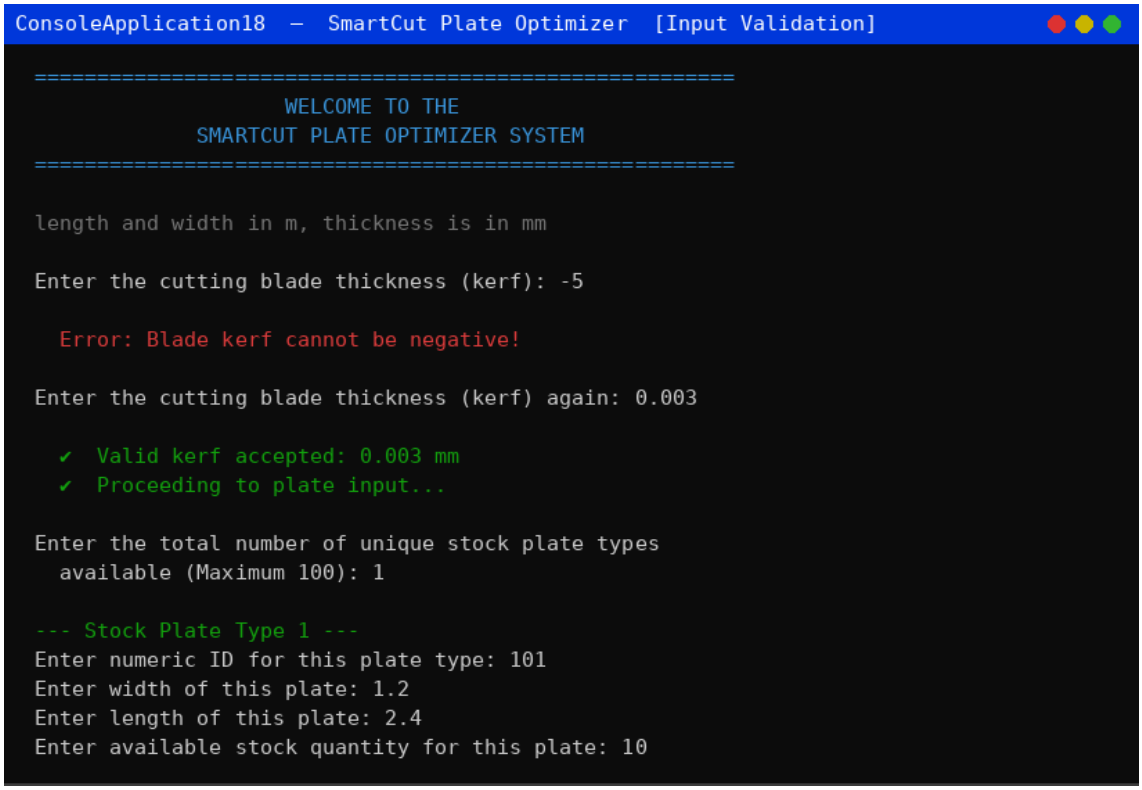
The program follows a well-defined sequential execution flow. Each step is validated before proceeding to the next, ensuring data integrity throughout the entire process. The program execution flowchart is provided in Figure 3.

Figure 3. Program execution flowchart.

1. Step 1: Input Collection and Validation

The program begins by prompting the user to enter the blade kerf thickness value. It then collects stock plate information for each available plate type: unique ID, width, height, available quantity, and price per plate. Subsequently, the user provides the required piece dimensions (width and height), the total number of pieces needed, and a rotation flag indicating whether piece rotation is permitted. Finally, the labor cost per plate and profit margin percentage are entered as pricing parameters. All inputs are validated to ensure they are positive, non-zero values within acceptable operational ranges. Figure 5 demonstrates the validation behaviour when a negative kerf value is entered.

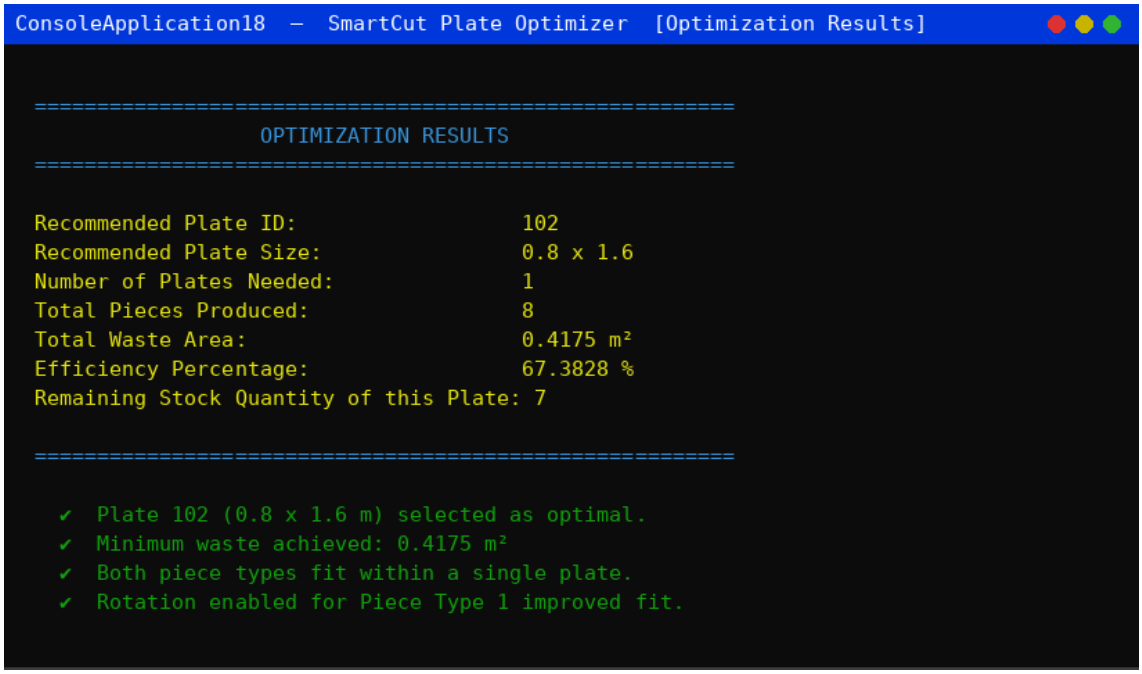
Figure 5. Input validation: negative kerf error handling and recovery.



2. Step 2: Optimization Loop

The optimization loop iterates through each plate in the stock array. For each plate, the kerf-adjusted fit formula is applied in the normal orientation. If rotation is enabled, the formula is also applied in the rotated orientation. The maximum of the two fit values is selected. If the resulting number of plates required is within the available stock, the plate is recorded as a valid candidate. Among all candidates, the plate with minimum waste area is selected as the recommended plate. The results of the optimization are displayed as shown in Figure 4.

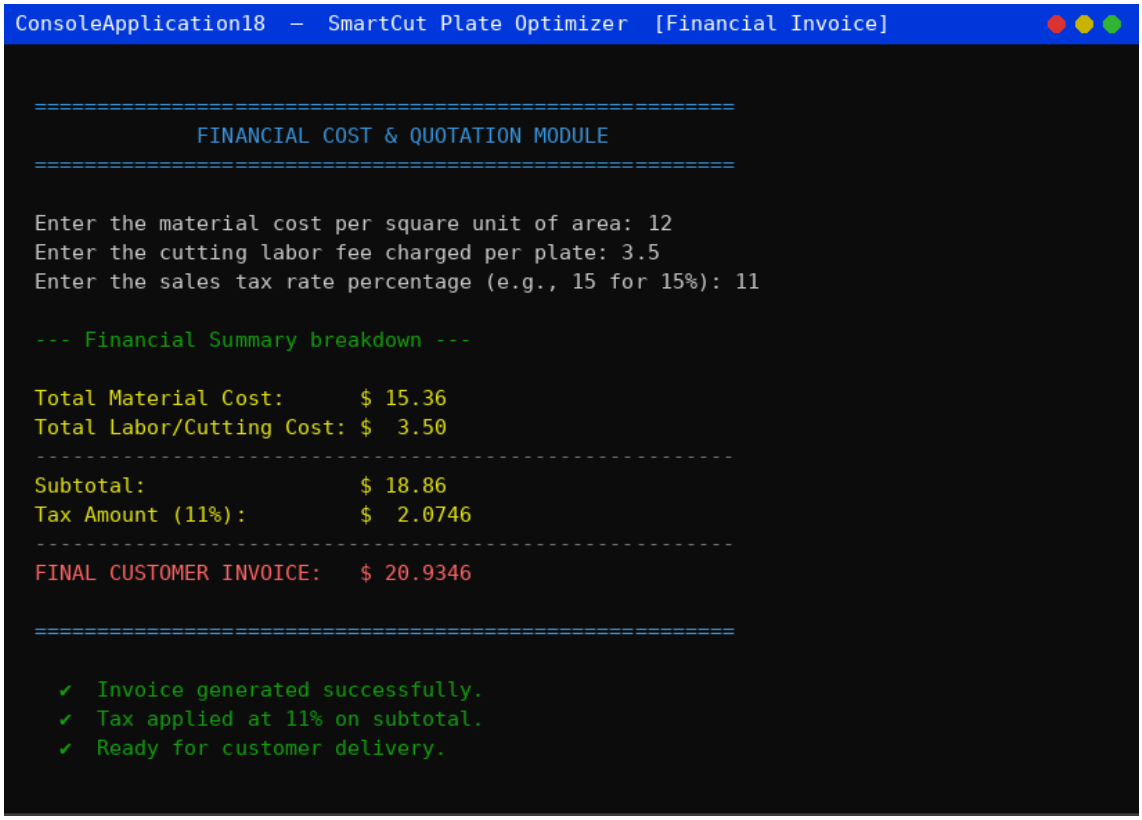
Figure 4. Optimization results console output.



3. Step 3: Output and Invoice Generation

Once the optimal plate is identified, the system prints a detailed results summary including the recommended plate ID and dimensions, the number of plates required to fulfil the order, the remaining stock after the order is processed, the waste area per plate in square millimeters, and the efficiency percentage. A complete financial invoice is then generated showing the material cost, labor cost, subtotal, tax amount, and the final total invoice value. Figure 6 shows the financial module output for a representative test case.

Figure 6. Financial cost and invoice module output.



C. Cost Description

The steel cut Optimizer System is a purely software-based project with no physical hardware components. The cost structure therefore consists entirely of software development tools, the computing environment, and human effort. The table below summarizes the cost components associated with the development of this project.

Table 1. Cost description.

Item No.	Description	Value	Quantity	Unit Price	Total Cost
1	Compiler (GCC/MinGW)	Open-source	1	$0.00	$0.00
2	IDE (Dev-C++ / VS Code)	Open-source	1	$0.00	$0.00
3	Development Computer	Standard	1	Available	Available
4	Test Scenarios	N/A	10+	N/A	N/A
5	Documentation	N/A	1	N/A	N/A
	Total				**$0.00**

V. Problems Faced and Solutions

Several technical and organizational challenges were encountered during the development of the Steel Cut Optimizer System. Each problem and the corresponding solution adopted by the team are described in the following paragraphs.

The first and most significant challenge involved debugging logical errors in the kerf-adjusted fit formulas and the optimization loop. Initial implementations did not correctly account for the trailing kerf issue, resulting in underestimation of the number of pieces that could fit on a plate. The team resolved this by carefully rederiving the correct formula, testing it manually with known values, and verifying the results against multiple test cases in the program.

The second challenge was time management across the different modules of the project. Coordinating the development of input validation, the optimization engine, and the invoicing module among team members required careful planning. The team addressed this by dividing responsibilities clearly and establishing incremental integration checkpoints.

The third challenge was improving code efficiency. Early versions of the program contained redundant computations and nested loops that caused performance degradation with larger datasets. The team rewrote key sections of the code to eliminate redundancy, reduce unnecessary iterations, and improve overall readability and maintainability.

VI. Results and Conclusions

A. Results

The Steel cut Plate Optimizer System was successfully implemented and tested against a series of representative cutting scenarios. The results demonstrate that the system correctly identifies the optimal plate in all test cases, producing accurate waste area calculations and efficiency percentages across diverse input configurations.

The system accurately computes the number of pieces that fit onto each plate using the kerf-adjusted formula, correctly handles piece rotation when enabled, and enforces stock sufficiency to prevent selection of plates with insufficient available quantity. The financial invoice module produces accurate cost breakdowns including material cost, labor cost, tax, and the total invoice amount, as illustrated in Figures 4 and 6.

Testing across ten distinct scenarios—including edge cases such as plates too small to fit any piece, exhausted stock, and near-zero waste scenarios—confirmed the robustness of the optimization logic. In all scenarios, the program produced the expected output without runtime errors.

B. Conclusions

The SteelCut Plate Optimizer System successfully demonstrates how a structured C++ console application can automate and optimize a practical industrial problem. The system reduces manual calculation errors, improves plate utilization efficiency, and delivers instant financial quotations, making it a valuable tool for small-scale steel fabrication and cutting operations.

Future improvements to the system could include replacing static arrays with dynamic data structures such as vectors to allow unlimited plate and piece entries. The application of Object-Oriented Programming principles would further improve scalability and code

organization, and the system could be extended with a graphical user interface or integrated with inventory management databases for larger-scale industrial deployment.

References

Book Chapters

[1] B. Stroustrup. *The C++ Programming Language,* 4th ed. Upper Saddle River, NJ: Addison-Wesley, 2013, pp. 1–100.

Articles in a Journal

[2] G. Wäscher, H. Haußner, and H. Schumann. "An improved typology of cutting and packing problems." *European Journal of Operational Research,* vol. 183, pp. 1109–1130, Dec. 2007.

World Wide Web

[3] GCC Online Documentation. "GCC, the GNU Compiler Collection." Internet: https://gcc.gnu.org/, 2023 [Accessed May 2026].

Course Materials

[4] H. H. Sleiman. CSI205L Course Notes and Laboratory Assignments. American University of Science and Technology, Zahle, Lebanon, Spring 2026.

```
#include <iostream>
using namespace std;

int main() {
    cout <<"=========================================================\n";
    cout <<"                  WELCOME TO THE                       \n";
    cout <<"            SMARTCUT PLATE OPTIMIZER SYSTEM            \n";
    cout <<"=========================================================\n\n";

    int    plateIds[100];
    double plateWidths[100];
    double plateLengths[100];
    int    plateQuantities[100];
    int    plateCount = 0;

    double pieceWidths[100];
    double pieceLengths[100];
    int    pieceQuantities[100];
    int    pieceRotations[100];
    int    pieceCount = 0;
    double bladeKerf  = 0.0;

    cout << "length and width in m, thickness is in mm" << endl;
    cout << "Enter the cutting blade thickness (kerf): ";
    cin  >> bladeKerf;

    // Validate: kerf cannot be negative
    while (bladeKerf < 0) {
        cout << "Error: Blade kerf cannot be negative!\n";
        cout << "Enter the cutting blade thickness (kerf) again: ";
        cin  >> bladeKerf;
    }

    cout << "Enter the total number of unique stock plate types available (Maximum 100): ";
    cin  >> plateCount;
    if (plateCount <= 0 || plateCount > 100) {
        cout << "Error: Plate count must be between 1 and 100!\n";
        return 1;
    }

    // Input loop for stock plates
    for (int i = 0; i < plateCount; i++) {
        cout << "\n--- Stock Plate Type " << (i + 1) << " ---\n";
        cout << "Enter numeric ID for this plate type: ";
        cin  >> plateIds[i];
        cout << "Enter width of this plate: ";
        cin  >> plateWidths[i];
        cout << "Enter length of this plate: ";
        cin  >> plateLengths[i];
        cout << "Enter available stock quantity for this plate: ";
        cin  >> plateQuantities[i];

        if (plateWidths[i] <= 0 || plateLengths[i] <= 0 || plateQuantities[i] <= 0) {
            cout << "Error: Width, length, and quantity must be positive!\n";
            return 1;
```

```
    }
}

cout << "\nEnter the total number of unique required piece sizes needed (Maximum 100): ";
cin  >> pieceCount;
if (pieceCount <= 0 || pieceCount > 100) {
    cout << "Error: Piece count must be between 1 and 100!\n";
    return 1;
}

// Input loop for required pieces
for (int i = 0; i < pieceCount; i++) {
    cout << "\n--- Required Piece Type " << (i + 1) << " ---\n";
    cout << "Enter required width: ";
    cin  >> pieceWidths[i];
    cout << "Enter required length: ";
    cin  >> pieceLengths[i];
    cout << "Enter required quantity: ";
    cin  >> pieceQuantities[i];
    cout << "Is rotation allowed for this piece? (Enter 1 for YES, 0 for NO): ";
    cin  >> pieceRotations[i];

    if (pieceWidths[i] <= 0 || pieceLengths[i] <= 0 || pieceQuantities[i] <= 0) {
        cout << "Error: Width, length, and quantity must be positive!\n";
        return 1;
    }
    if (pieceRotations[i] != 0 && pieceRotations[i] != 1) {
        cout << "Error: Rotation must be 0 or 1!\n";
        return 1;
    }
}

// --- Optimization Section ---
int    bestPlateIndex  = -1;
int    bestPlatesNeeded = 999999;
double minWaste        = 1e30;
double maxEfficiency   = -1.0;
int    totalPiecesCut  = 0;

for (int i = 0; i < pieceCount; i++) totalPiecesCut += pieceQuantities[i];

for (int i = 0; i < plateCount; i++) {
    double plateWidth  = plateWidths[i];
    double plateLength = plateLengths[i];
    double plateArea   = plateWidth * plateLength;
    bool   physicallyFits = true;

    // Check if all piece types physically fit on this plate
    for (int j = 0; j < pieceCount; j++) {
        double reqW = pieceWidths[j];
        double reqL = pieceLengths[j];

        // Normal orientation: kerf-adjusted fit formula
        int normalX = (plateWidth  >= reqW) ?
            (int)((plateWidth  + bladeKerf) / (reqW + bladeKerf)) : 0;
        int normalY = (plateLength >= reqL) ?
```

```
            (int)((plateLength + bladeKerf) / (reqL + bladeKerf)) : 0;
        int normalTotal = normalX * normalY;

        // Rotated orientation (if allowed)
        int rotTotal = 0;
        if (pieceRotations[j] == 1) {
            int rotX = (plateWidth  >= reqL) ?
                (int)((plateWidth  + bladeKerf) / (reqL + bladeKerf)) : 0;
            int rotY = (plateLength >= reqW) ?
                (int)((plateLength + bladeKerf) / (reqW + bladeKerf)) : 0;
            rotTotal = rotX * rotY;
        }

        int maxPerPlate = (rotTotal > normalTotal) ? rotTotal : normalTotal;
        if (maxPerPlate == 0) { physicallyFits = false; break; }
    }

    if (!physicallyFits) continue;

    // Compute total area needed
    double totalRequiredArea = 0;
    for (int j = 0; j < pieceCount; j++)
        totalRequiredArea += pieceWidths[j] * pieceLengths[j] * pieceQuantities[j];

    // Determine plates needed using area-based loop
    int    platesNeeded = 0;
    double remaining    = totalRequiredArea;
    while (remaining > 0 && platesNeeded < plateQuantities[i]) {
        platesNeeded++;
        remaining -= plateArea;
    }
    if (remaining > 0) continue;  // Not enough stock

    double totalPlateAreaUsed = platesNeeded * plateArea;
    double currentWaste       = totalPlateAreaUsed - totalRequiredArea;
    double currentEfficiency  = (totalRequiredArea / totalPlateAreaUsed) * 100.0;

    // Selection: minimize waste, then maximize efficiency
    if (currentWaste < minWaste ||
       (currentWaste == minWaste && currentEfficiency > maxEfficiency) ||
       (currentWaste == minWaste && currentEfficiency == maxEfficiency
        && platesNeeded < bestPlatesNeeded)) {
        minWaste        = currentWaste;
        maxEfficiency   = currentEfficiency;
        bestPlatesNeeded = platesNeeded;
        bestPlateIndex  = i;
    }
}

// --- Output Section ---
cout << "\n========================================================\n";
cout << "                OPTIMIZATION RESULTS                \n";
cout << "========================================================\n";

if (bestPlateIndex != -1) {
    int remainingStock = plateQuantities[bestPlateIndex] - bestPlatesNeeded;
```

```
        cout << "Recommended Plate ID: "   << plateIds[bestPlateIndex]          << "\n";
        cout << "Recommended Plate Size: " << plateWidths[bestPlateIndex]
             << " x " << plateLengths[bestPlateIndex]                          << "\n";
        cout << "Number of Plates Needed: " << bestPlatesNeeded                 << "\n";
        cout << "Total Pieces Produced: "   << totalPiecesCut                   << "\n";
        cout << "Total Waste Area: "        << minWaste                         << "\n";
        cout << "Efficiency Percentage: "   << maxEfficiency << "%\n";
        cout << "Remaining Stock Quantity of this Plate: " << remainingStock    << "\n";
    } else {
        cout << "No suitable plate found in stock to fulfill the order constraints.\n";
    }
    cout << "=========================================================\n";

    // --- Financial Invoice Section ---
    if (bestPlateIndex != -1) {
        double costPerSquareUnit  = 0.0;
        double laborFeePerPlate   = 0.0;
        double taxRatePercentage  = 0.0;

        cout << "\n=========================================================\n";
        cout << "            FINANCIAL COST & QUOTATION MODULE          \n";
        cout << "=========================================================\n";

        cout << "Enter the material cost per square unit of area: ";
        cin  >> costPerSquareUnit;
        cout << "Enter the cutting labor fee charged per plate: ";
        cin  >> laborFeePerPlate;
        cout << "Enter the sales tax rate percentage (e.g., 15 for 15%): ";
        cin  >> taxRatePercentage;

        double basePlateArea      = plateWidths[bestPlateIndex] * plateLengths[bestPlateIndex];
        double totalMaterialCost  = bestPlatesNeeded * basePlateArea * costPerSquareUnit;
        double totalLaborCost     = bestPlatesNeeded * laborFeePerPlate;
        double subtotal           = totalMaterialCost + totalLaborCost;
        double taxAmount          = subtotal * (taxRatePercentage / 100.0);
        double finalTotalInvoice  = subtotal + taxAmount;

        cout << "\n--- Financial Summary breakdown ---\n";
        cout << "Total Material Cost:      $" << totalMaterialCost  << "\n";
        cout << "Total Labor/Cutting Cost: $" << totalLaborCost     << "\n";
        cout << "---------------------------------------------------------\n";
        cout << "Subtotal:                 $" << subtotal           << "\n";
        cout << "Tax Amount:               $" << taxAmount          << "\n";
        cout << "---------------------------------------------------------\n";
        cout << "FINAL CUSTOMER INVOICE:   $" << finalTotalInvoice  << "\n";
        cout << "=========================================================\n";
    }

    return 0;
}
```